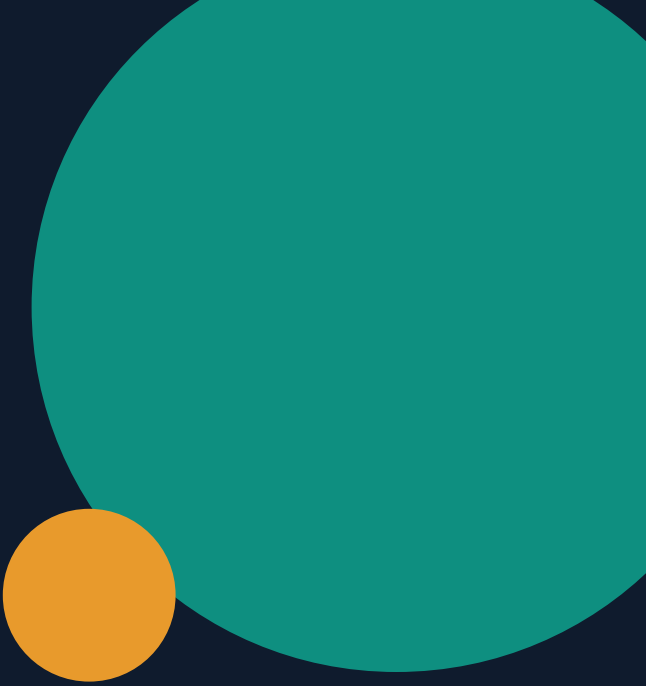




OPERATIONS PLAYBOOK

Playwright Autopilot

Agentic end-to-end testing plugin for Claude Code

Install · Configure · Execute · Integrate with CI · Troubleshoot

For QA engineers, SDETs, developers and plugin maintainers

Contents

1. How to use this playbook	4
Conventions	4
2. What the plugin is	4
The reasoning lifecycle	5
Components at a glance	6
3. Prerequisites	6
4. Publish the plugin to GitHub (maintainers)	7
Releasing a new version	7
5. Install the plugin (users)	7
Verify the installation	8
Alternative install paths	8
6. Prepare your test project	8
Configure environment variables	9
7. Run the end-to-end lifecycle	9
What you will see, phase by phase	9
Where the output lives	10
Running the generated tests yourself	10
Resuming an interrupted run	10
8. Skill reference	10
Helper CLIs	11
9. Quality guardrails	11
Failure taxonomy	11
What the healer is never allowed to do	12
Blocked by the spec-guard hook	12
10. CI/CD integration	12
11. Configuration	12
12. Troubleshooting	13
12.1 Installation and marketplace	13
12.2 Playwright MCP server	13
12.3 Helper CLIs and hook	13
12.4 Project setup	14

12.5 Test execution	14
12.6 Agent behaviour	15
12.7 Windows specifics	15
12.8 CI pipeline	15
12.9 Diagnostic procedure	15
13. Operational checklists	16
First-run checklist	16
Pull-request checklist for generated tests	16
Safe-use principles	16

1. How to use this playbook

This playbook is the single reference for installing, running, integrating and troubleshooting **Playwright Autopilot**, a Claude Code plugin that plans, explores, writes, runs, heals and reviews Playwright end-to-end tests. Follow the sections in order the first time; afterwards jump straight to the part you need.

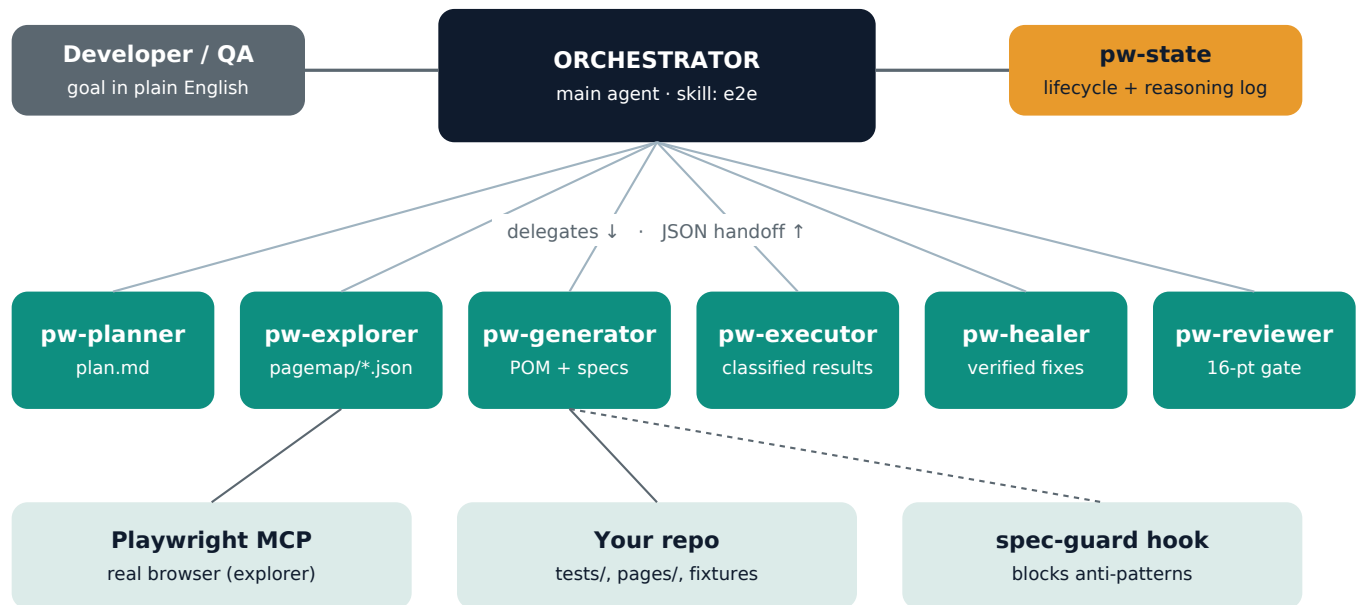
If you are...	Go to	Outcome
A maintainer publishing the plugin	Section 4	Repo on GitHub, installable by anyone
A first-time user	Sections 3, 5, 6	Plugin installed and verified in 10 minutes
Writing tests for a feature	Sections 7, 8	Reviewed, passing Playwright suite + report
Setting up pipelines	Section 10	Sharded GitHub Actions / Azure / GitLab run
Stuck on an error	Section 12	Symptom → cause → fix table

Conventions

- Monospace text is a command or file path. Lines starting with / are typed **inside** Claude Code; everything else runs in a normal terminal.
- YOUR_GITHUB_USER is the GitHub account or organisation hosting the plugin.
- Commands are shown for macOS/Linux shells. Windows differences are called out explicitly (Claude Code on Windows runs Bash commands through Git Bash).

2. What the plugin is

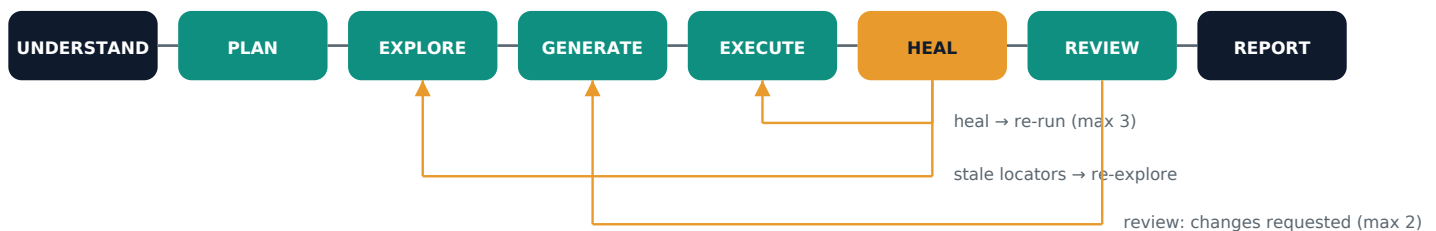
Playwright Autopilot turns Claude Code into a small, disciplined QA team. A main **orchestrator** owns the workflow and delegates focused work to six specialist **subagents**. Each subagent works in its own context window with only the tools it needs and returns a structured JSON handoff that the orchestrator validates before moving on.



The reasoning lifecycle

Every run moves through eight phases. At each phase exit the orchestrator records an **observation** → **hypothesis** → **decision** → **confidence** entry, producing an audit trail anyone can read after the run.

Legal transitions are enforced by pw-state (illegal jump → exit 4; heal budget spent → exit 3)



Phase	Owner	Exit gate
UNDERSTAND	Orchestrator	Target URL, environment, auth method and scope known (or assumptions stated)
PLAN	pw-planner	plan.md exists with ≥ 1 P0 scenario and no blocking questions
EXPLORE	pw-explorer (up to 4 in parallel)	Every element needed by P0 scenarios verified against the live app
GENERATE	pw-generator	playwright test --list succeeds; every P0 ID traced to a test
EXECUTE	pw-executor	Results run with retries=0 and failures classified
HEAL	pw-healer	Fixes verified, heal budget spent, or only app bugs / environment issues remain
REVIEW	pw-reviewer	Score $\geq 13/16$ with no zero on traceability, assertions or security
REPORT	Orchestrator	report.md written and summary delivered

Components at a glance

Component	Location	Purpose
Skills (11)	skills/*/SKILL.md	Entry points (e2e, setup, explore, generate, heal, review, api-testing, ci, status) and reference knowledge (locators, pom-fixtures)
Subagents (6)	agents/pw-*.md	Specialists with scoped tools and JSON handoff contracts
Helper CLIs (3)	bin/pw-detect, pw-state, pw-results	Project detection, lifecycle state machine, test-result digest; on PATH while the plugin is enabled
Guardrail hook	hooks/spec-guard.mjs	Rejects anti-patterns every time a spec or page object is written
MCP server	.mcp.json	Starts @playwright/mcp so the explorer can drive a real browser
Manifests	.claude-plugin/plugin.json, marketplace.json	Make the repo installable as a plugin and a marketplace

3. Prerequisites

Requirement	Version / detail	Check with
Claude Code	Current release, signed in	claude --version
Node.js	18 or later (20 LTS recommended)	node -v
npm / npx	Bundled with Node	npx -v
Git	Any recent version; Git Bash on Windows	git --version
Network	Access to github.com, registry.npmjs.org, Playwright browser CDN	npm ping
App under test	Reachable URL (local, dev or staging) and test credentials	Open it in a browser
Disk	~1 GB free for browser binaries	—

IMPORTANT Use a non-production environment and dedicated test accounts. The explorer interacts with the live app, and generated tests may create data.

4. Publish the plugin to GitHub (maintainers)

- 1 Unzip playwright-autopilot.zip and open the folder.
- 2 Replace YOUR_GITHUB_USER in README.md and .claude-plugin/plugin.json.
- 3 Run the structural validator; it must print *All checks passed*.
- 4 Create an empty repository named playwright-autopilot on GitHub (no README, no licence — the repo already has them).
- 5 Commit and push as shown below.

Terminal — from the plugin folder

```
node tests/validate.mjs

git init
git add .
git update-index --chmod=+x bin/pw-state bin/pw-results bin/pw-detect hooks/spec-guard.mjs
git commit -m "feat: playwright-autopilot v1.0.0"
git branch -M main
git remote add origin https://github.com/YOUR_GITHUB_USER/playwright-autopilot.git
git push -u origin main
```

IMPORTANT The `git update-index --chmod=+x` line matters when you commit from Windows: without it the helper CLIs lose their executable bit and Mac/Linux users see "permission denied". `.gitattributes` already forces LF line endings so the shebang lines keep working.

Confirm the **Validate plugin** GitHub Action goes green on the first push. It checks manifests, frontmatter, script syntax and runs a lifecycle smoke test.

Releasing a new version

- 1 Make changes and run `node tests/validate.mjs`.
- 2 Bump version in **both** `plugin.json` and `marketplace.json` (the validator fails if they differ).
- 3 Commit, tag (`git tag v1.1.0`) and push with tags.
- 4 Tell users to run `/plugin marketplace update playwright-autopilot`.

5. Install the plugin (users)

Open Claude Code in any folder and run the two commands below. The first registers the GitHub repo as a *marketplace*; the second installs the plugin from it using the format `plugin@marketplace`.

Inside Claude Code

```
/plugin marketplace add YOUR_GITHUB_USER/playwright-autopilot
/plugin install playwright-autopilot@playwright-autopilot
```

Restart Claude Code after installing so subagents, hooks and the MCP server are loaded.

Verify the installation

Check	Command	Expected result
Plugin enabled	<code>/plugin</code>	playwright-autopilot listed as installed and enabled
Subagents loaded	<code>/agents</code>	pw-planner, pw-explorer, pw-generator, pw-executor, pw-healer, pw-reviewer
MCP connected	<code>/mcp</code>	<i>playwright</i> server connected (first start downloads the package, ~30-60 s)
Helper CLIs	Ask Claude: "run pw-detect"	JSON describing the current project
Skills visible	Type <code>/playwright-autopilot:</code>	Autocomplete lists <code>e2e</code> , <code>setup</code> , <code>heal</code> , <code>review</code> ...

Alternative install paths

Scenario	How
Try it before publishing	<code>claude --plugin-dir ./playwright-autopilot</code>
Private GitHub repo	Works if your local git credentials (SSH key or credential manager) can clone the repo
Whole team, automatically	Add the marketplace and plugin to the project's <code>.claude/settings.json</code> (<code>extraKnownMarketplaces</code> / <code>enabledPlugins</code>). Teammates are prompted to install when they trust the folder. Confirm key names in the current Claude Code settings docs.
Pin a version	Install from a tag or fork the repo at the release you want

6. Prepare your test project

Always start Claude Code from the **root of the project you want to test** — the plugin reads and writes relative to the current folder.

Terminal, then Claude Code

```
cd my-web-app
claude
/playwright-autopilot:setup ts
```

The **setup** skill runs `pw-detect`, then installs `@playwright/test` and `dotenv`, downloads Chromium, and adds the following (it asks before touching an existing app repo and never overwrites your config without showing a diff):

File	What it gives you
<code>playwright.config.ts</code>	baseURL from env, traces/screens/video on failure, CI-aware retries and reporters, optional auth setup project
<code>tests/fixtures.ts</code>	Typed <code>test.extend()</code> for injecting page objects
<code>tests/pages/BasePage.ts</code>	<code>goto()</code> + <code>readySignal</code> pattern every page object inherits
<code>tests/auth.setup.ts</code>	Logs in once and saves <code>storageState</code> for all tests (only if auth is needed)
<code>.env.example</code>	<code>BASE_URL</code> , <code>E2E_USER</code> , <code>E2E_PASSWORD</code> , <code>TEST_ID_ATTR</code>

File	What it gives you
.gitignore additions	test-results/, playwright-report/, playwright/.auth/, .env

Configure environment variables

.env (never commit this file)

```
cp .env.example .env
# edit .env
BASE_URL=https://staging.myapp.com
E2E_USER=qa.bot@myapp.com
E2E_PASSWORD=*****
```

DO NOT Never paste passwords into the chat. Agents only reference variable *names* (e.g. \$E2E_USER); the spec-guard hook rejects files containing hard-coded credentials.

7. Run the end-to-end lifecycle

Give the orchestrator a goal: a URL plus the behaviour to cover, a user story, or a path to a requirements file.

Inside Claude Code

```
/playwright-autopilot:e2e https://staging.myapp.com/login login with valid, invalid and locked users

# or reference a spec document
/playwright-autopilot:e2e docs/stories/checkout-coupons.md

# or just ask in plain English
write playwright tests for the signup page including validation errors
```

What you will see, phase by phase

Phase	On screen	Your action
UNDERSTAND	One-line assumptions, or one clarifying question if something blocking is missing	Answer the question if asked
PLAN	"► PLAN: 9 scenarios, 4 P0"; table shown if more than 10 scenarios	Object now if scope is wrong; otherwise let it continue
EXPLORE	Parallel explorers navigate the app; pagemap files appear	Approve MCP browser tool permissions when prompted
GENERATE	Files created under tests/; spec-guard may bounce a file back	None — fixes are automatic
EXECUTE	Pass/fail counts with root-cause categories	Approve the Bash command if permission prompts are on
HEAL	Per-failure root cause, change and verification	Decide whether confirmed app bugs are marked test.fail()
REVIEW	Score out of 16 and verdict	None — rework loops automatically (max 2)

Phase	On screen	Your action
REPORT	Summary, files, traceability, app bugs, run commands	Review the diff and commit

Where the output lives

```
tests/
  pages/      LoginPage.ts, DashboardPage.ts ...
  fixtures.ts
  e2e/        login.spec.ts ...
.pw-autopilot/runs/<runId>/
  state.json  phase, history, heal budget, artifacts
  reasoning.md decision trail (observation / hypothesis / decision / confidence)
  plan.md     scenarios with IDs, priorities, expected results
  pagemap/    verified locators per page (JSON)
  review.md   quality scorecard
  report.md   final summary
```

Running the generated tests yourself

```
npx playwright test                # everything
npx playwright test --grep @smoke  # tagged subset
npx playwright test tests/e2e/login.spec.ts --headed
npx playwright test --ui           # interactive runner
npx playwright show-report         # HTML report
```

Resuming an interrupted run

Run `/playwright-autopilot:status`. It shows the current phase and artifacts, and continues from that phase without re-planning or re-exploring verified work.

8. Skill reference

Skill	Use it when	Example
e2e	You want new tests for a feature or journey, fully planned and reviewed	<code>/playwright-autopilot:e2e https://app/cart apply coupon</code>
setup	Project has no Playwright, or config is incomplete	<code>/playwright-autopilot:setup ts</code>
explore	You only need verified locators / a page map	<code>/playwright-autopilot:explore https://app/settings</code>
generate	Quick single scenario; converting Selenium, Cypress or Gherkin	<code>/playwright-autopilot:generate convert tests/legacy/LoginTest.java</code>
heal	Existing tests fail or are flaky after UI changes	<code>/playwright-autopilot:heal tests/e2e/checkout.spec.ts</code>
review	Audit a suite or a pull request for quality	<code>/playwright-autopilot:review tests/e2e</code>
api-testing	API, contract, hybrid UI+API, or network mocking	<code>/playwright-autopilot:api-testing openapi.yaml</code>

Skill	Use it when	Example
ci	Run the suite in a pipeline	/playwright-autopilot:ci github 4
status	See progress, decisions, or resume	/playwright-autopilot:status
locators, pom-fixtures	Loaded automatically as reference while writing code	—

Helper CLIs

Command	Purpose
pw-detect	Prints language, package manager, config path, testDir, baseUrl, POM/fixture usage and existing specs
pw-state init "goal"	Starts a run and prints its runId
pw-state status [runId]	Current phase, allowed next phases, heal budget, artifacts, last reasoning entry
pw-state advance PHASE	Moves to a phase; exits 4 on an illegal transition
pw-state reason PHASE '{json}'	Appends a reasoning record to state.json and reasoning.md
pw-state heal-attempt	Consumes one heal attempt; exits 3 when the budget is spent
pw-results [file]	Turns a Playwright JSON report into a compact, categorised failure digest

9. Quality guardrails

Failure taxonomy

Category	Typical signal	Healer strategy
LOCATOR_NOT_FOUND	waiting for getByRole(...) / element not found	Check name change, iframe, page state; re-explore if confidence < 0.7
LOCATOR_AMBIGUOUS	strict mode violation: resolved to N elements	Scope to container, filter, or exact: true
ASSERTION_MISMATCH	Expected "X" Received "Y"	Regex/computed value for dynamic data; APP_BUG if a real regression
TIMEOUT	Test timeout exceeded	Add proper ready signal; await navigation
ENVIRONMENT	ECONNREFUSED, net::ERR_*	No test edits; report to the user
CODE_ERROR	TypeError, SyntaxError	Fix the code
APP_BUG	Evidence the app contradicts the plan	Report expected vs actual; never mask

What the healer is never allowed to do

- Delete or weaken assertions (e.g. replace `toHaveText` with `toBeVisible`).
- Add `waitForTimeout` or raise timeouts beyond 2× the default.
- Use force: true clicks or `test.skip` without a documented reason.

Blocked by the spec-guard hook

`waitForTimeout` · `page.$` / `page.$$` · `setTimeout` · absolute or positional XPath · `nth-child` / `nth-of-type` · `expect(await x.isVisible())` · `test.only` / `describe.only` · hard-coded password/secret/token literals.

10. CI/CD integration

Inside Claude Code

```
/playwright-autopilot:ci github 4
```

This writes `.github/workflows/playwright.yml` with four parallel shards, browser install, blob reports and a merge job that publishes one HTML report. Then:

- 1 In GitHub → Settings → Secrets and variables → Actions, create secrets `E2E_USER` and `E2E_PASSWORD` and a variable `BASE_URL`.
- 2 Push the workflow. It runs on pull requests, pushes to main, nightly at 03:00 UTC and on demand.
- 3 Download *playwright-report* from the run's artifacts to view results.
- 4 When CI fails, download the JSON results and run `/playwright-autopilot:heal path/to/results.json`.

Azure DevOps, GitLab CI and Jenkins are supported by passing `azure`, `gitlab` or `jenkins` instead of `github`.

11. Configuration

Setting	Default	Effect
PW_AUTOPILOT_MAX_HEAL	3	Heal iterations before the orchestrator stops healing and escalates
BASE_URL	http://localhost:3000	Application under test (read by config and agents)
E2E_USER / E2E_PASSWORD	—	Credentials for the auth setup project and explorer
TEST_ID_ATTR	data-testid	Attribute used by <code>getByTestId</code> (e.g. <code>data-qa</code> , <code>data-cy</code>)
Browsers	chromium only	Enable firefox / webkit / mobile projects in <code>playwright.config.ts</code> once stable
Subagent models	inherit; executor uses haiku	Edit model: in <code>agents/*.md</code> to trade cost for depth

12. Troubleshooting

Find the symptom, confirm the likely cause, apply the fix. If nothing matches, use the diagnostic procedure in 12.9.

12.1 Installation and marketplace

Symptom	Likely cause	Fix
/plugin marketplace add fails: repository not found	Typo in owner/repo, or private repo without credentials	Check the URL in a browser; run git clone of the repo manually to confirm access
"No marketplace.json found"	.claude-plugin/marketplace.json missing from repo root	Ensure the .claude-plugin folder was committed (hidden folders are sometimes skipped by upload tools)
install fails: plugin not found in marketplace	Wrong name after @	Use playwright-autopilot@playwright-autopilot exactly
Skills missing from autocomplete	Claude Code not restarted after install	Restart; check /plugin shows it enabled
Old behaviour after an update	Marketplace cache not refreshed / version not bumped	/plugin marketplace update playwright-autopilot; maintainers bump both version fields

12.2 Playwright MCP server

Symptom	Likely cause	Fix
/mcp shows playwright failed or disconnected	npx could not download @playwright/mcp (proxy, offline)	Run npx -y @playwright/mcp@latest --help in a terminal; configure npm proxy (npm config set proxy ...)
Explorer says browser executable missing	Browsers not installed for the MCP package	npx playwright install chromium
Explorer falls back to probe script	MCP tools not permitted or not loaded	Approve MCP tool permissions; verify /mcp; restart
Explorer hits login wall or captcha	Auth not configured / bot protection	Set E2E_USER/E2E_PASSWORD; use a test env with captcha disabled or a test bypass

12.3 Helper CLIs and hook

Symptom	Likely cause	Fix
pw-state: command not found	Plugin disabled or older Claude Code without bin/ PATH support	Check /plugin; update Claude Code; fallback: node ~/.claude/plugins/.../bin/pw-state
permission denied: pw-state	Executable bit lost in commit	Maintainer runs git update-index --chmod=+x on bin files and pushes
/usr/bin/env: 'node\r': No such file	CRLF line endings from a Windows commit	Keep the provided .gitattributes; git add --renormalize . and push

Symptom	Likely cause	Fix
pw-state exit 4 "illegal transition"	A phase was skipped	Working as designed; run pw-state status and advance through an allowed phase
pw-state exit 3 "heal budget exhausted"	Healing reached PW_AUTOPILOT_MAX_HEAL	Review remaining failures; raise the variable only if fixes were progressing
Hook keeps rejecting a file	An anti-pattern is present (message lists which)	Let Claude fix it; if a match is a false positive, adjust the regex in hooks/spec-guard.mjs

12.4 Project setup

Symptom	Likely cause	Fix
Cannot find module '@playwright/test'	Dependency not installed in this project	/playwright-autopilot:setup or npm i -D @playwright/test
Executable doesn't exist at ...chromium	Browsers not downloaded	npx playwright install chromium (add --with-deps on Linux)
Host system is missing dependencies (Linux)	OS libraries for browsers missing	sudo npx playwright install-deps
Tests use wrong URL	BASE_URL not loaded	Confirm .env exists and config imports dotenv/config
auth.setup.ts fails	Login locators differ from template defaults	Run /playwright-autopilot:explore on the login page and update auth.setup.ts

12.5 Test execution

Symptom	Likely cause	Fix
ECONNREFUSED / net::ERR_CONNECTION_REFUSED	App not running or wrong port	Start the app or enable webServer in config; classified ENVIRONMENT, heal budget untouched
Passes locally, fails in CI	Missing secrets, slower CI, different data	Check repo secrets; use web-first assertions; seed data via API fixtures
Flaky test	Race with async UI, shared data between tests	/playwright-autopilot:heal with --repeat-each=5 reproduction; unique data per test
strict mode violation	Locator matches several elements	Healer scopes the locator; see locators skill
pw-results: JSON parse error	Console output mixed into the JSON report	Rerun with stderr redirected; pw-results already strips leading noise

12.6 Agent behaviour

Symptom	Likely cause	Fix
Planner asks for input repeatedly	No URL or spec provided	Give a URL or a requirements file path in the goal
Generated code ignores repo conventions	No existing specs to learn from / mixed styles	Point to a reference spec in the goal: "follow tests/e2e/orders.spec.ts style"
Run is slow or costly	Large scope, many pages explored	Split goals by feature; use generate for single scenarios; keep executor on haiku
Reviewer keeps requesting changes	Real quality gaps	Read review.md; after 2 loops remaining items are listed in the report for manual follow-up
Context seems lost mid-run	Very long session	Run <code>/playwright-autopilot:status</code> to resume from state on disk

12.7 Windows specifics

Symptom	Likely cause	Fix
Bash commands fail on Windows	Git Bash not installed	Install Git for Windows; Claude Code uses Git Bash for shell commands
Path errors with backslashes	Mixed path styles	Use forward slashes in goals and file references
playwright install --with-deps errors	--with-deps is Linux-only	Use <code>npx playwright install chromium</code>

12.8 CI pipeline

Symptom	Likely cause	Fix
Shard jobs pass but no merged report	merge-reports job skipped or artifact names differ	Keep blob-report-* naming; ensure merge job needs: [test]
Secrets empty in forked PRs	GitHub hides secrets from fork PRs	Run E2E on pushes/internal PRs, or use a pull_request_target pattern with care
Browser install slow	Downloading every run	Install chromium only; cache ~/.cache/ms-playwright keyed on Playwright version

12.9 Diagnostic procedure

- 1 Run `/plugin`, `/agents`, `/mcp` — confirm all three components are healthy.
- 2 Ask Claude to run `pw-detect` and `pw-state status`; read the phase and artifacts.
- 3 Open `.pw-autopilot/runs/<runId>/reasoning.md` — the last decisions usually explain the stop.
- 4 Reproduce outside the agent: `npx playwright test <file> --retries=0 --reporter=line`.
- 5 Inspect `test-results/*/error-context.md`, screenshots, or `npx playwright show-trace <trace.zip>`.

- 6 Still stuck: open a GitHub issue with Claude Code version, OS, Node version, the pw-state status output and the failing error.

13. Operational checklists

First-run checklist

✓	Item
☐	Node 18+, Git and Claude Code installed
☐	Plugin installed; /plugin, /agents, /mcp healthy
☐	Started Claude Code from the project root
☐	setup skill completed; npx playwright test --list works
☐	.env created and in .gitignore; no secrets in chat
☐	Non-production environment and test accounts
☐	First e2e run completed; report.md reviewed
☐	Generated diff reviewed before commit

Pull-request checklist for generated tests

✓	Item
☐	Review verdict approved (score $\geq 13/16$)
☐	Every P0 scenario ID appears in a test title
☐	No test.only, no waitForTimeout, no secrets
☐	App bugs raised as tickets with expected vs actual
☐	Suite passes locally with --retries=0
☐	CI workflow green

Safe-use principles

- Keep a human reviewer on every generated diff before merging.
- Run against test environments with synthetic data only.
- Treat reasoning.md and report.md as the audit record for what was automated and why.